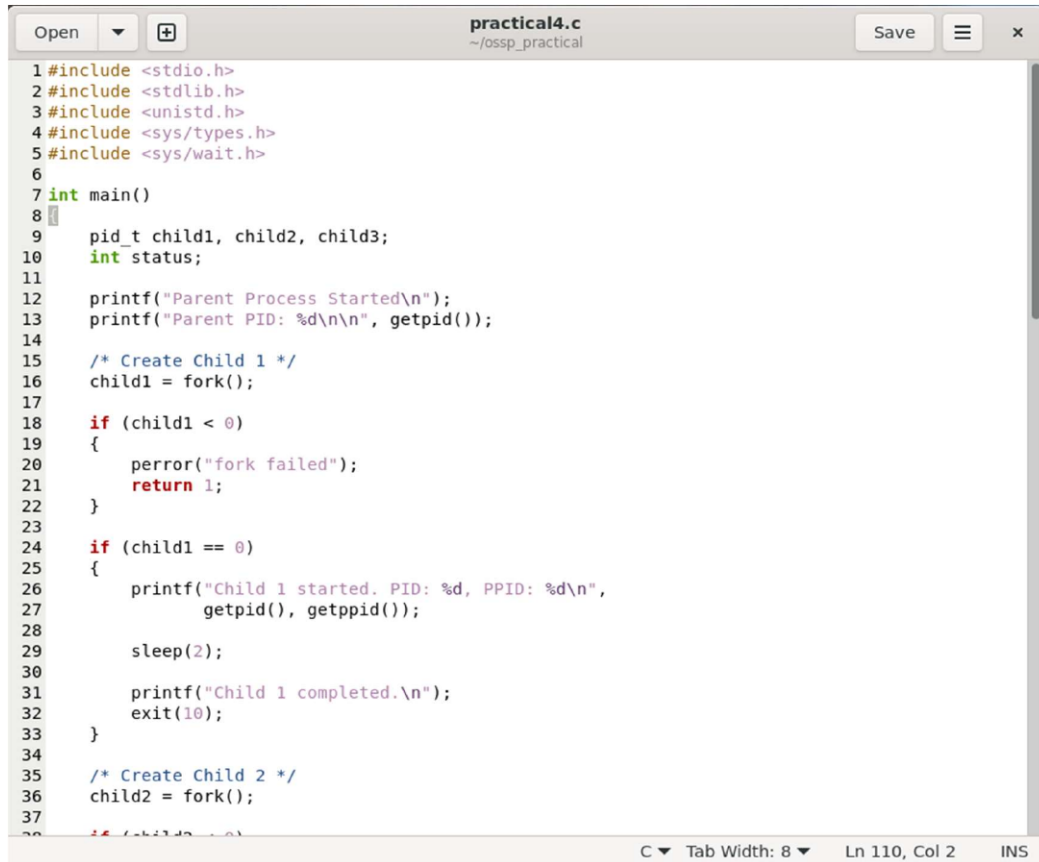
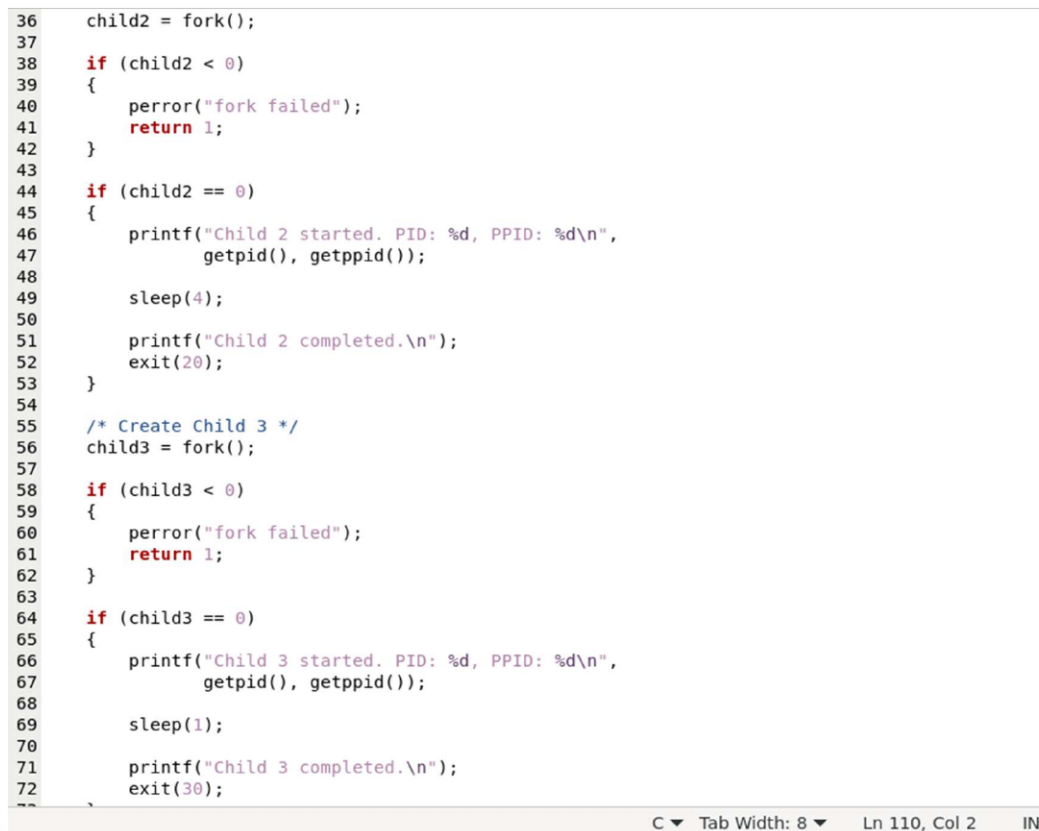


Practical – 4 – OS



```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/types.h>
5 #include <sys/wait.h>
6
7 int main()
8 {
9     pid_t child1, child2, child3;
10    int status;
11
12    printf("Parent Process Started\n");
13    printf("Parent PID: %d\n\n", getpid());
14
15    /* Create Child 1 */
16    child1 = fork();
17
18    if (child1 < 0)
19    {
20        perror("fork failed");
21        return 1;
22    }
23
24    if (child1 == 0)
25    {
26        printf("Child 1 started. PID: %d, PPID: %d\n",
27              getpid(), getppid());
28
29        sleep(2);
30
31        printf("Child 1 completed.\n");
32        exit(10);
33    }
34
35    /* Create Child 2 */
36    child2 = fork();
37
38    if (child2 < 0)
39    {
40        perror("fork failed");
41        return 1;
42    }
43
44    if (child2 == 0)
45    {
46        printf("Child 2 started. PID: %d, PPID: %d\n",
47              getpid(), getppid());
48
49        sleep(4);
50
51        printf("Child 2 completed.\n");
52        exit(20);
53    }
54
55    /* Create Child 3 */
56    child3 = fork();
57
58    if (child3 < 0)
59    {
60        perror("fork failed");
61        return 1;
62    }
63
64    if (child3 == 0)
65    {
66        printf("Child 3 started. PID: %d, PPID: %d\n",
67              getpid(), getppid());
68
69        sleep(1);
70
71        printf("Child 3 completed.\n");
72        exit(30);
73    }
74 }
```



```
36    child2 = fork();
37
38    if (child2 < 0)
39    {
40        perror("fork failed");
41        return 1;
42    }
43
44    if (child2 == 0)
45    {
46        printf("Child 2 started. PID: %d, PPID: %d\n",
47              getpid(), getppid());
48
49        sleep(4);
50
51        printf("Child 2 completed.\n");
52        exit(20);
53    }
54
55    /* Create Child 3 */
56    child3 = fork();
57
58    if (child3 < 0)
59    {
60        perror("fork failed");
61        return 1;
62    }
63
64    if (child3 == 0)
65    {
66        printf("Child 3 started. PID: %d, PPID: %d\n",
67              getpid(), getppid());
68
69        sleep(1);
70
71        printf("Child 3 completed.\n");
72        exit(30);
73    }
74 }
```

Practical – 4 – OS

```
73 }
74
75 /* Parent waits using wait() */
76 printf("\nParent waiting using wait()...\n");
77
78 wait(&status);
79
80 if (WIFEXITED(status))
81 {
82     printf("wait(): A child terminated with exit status %d\n",
83           WEXITSTATUS(status));
84 }
85
86 /* Parent waits for a specific child using waitpid() */
87 printf("\nParent waiting for Child 2 using waitpid()...\n");
88
89 waitpid(child2, &status, 0);
90
91 if (WIFEXITED(status))
92 {
93     printf("waitpid(): Child 2 terminated with exit status %d\n",
94           WEXITSTATUS(status));
95 }
96
97 /* Wait for remaining child */
98 waitpid(child1, &status, 0);
99
100 if (WIFEXITED(status))
101 {
102     printf("Child 1 terminated with exit status %d\n",
103           WEXITSTATUS(status));
104 }
105
106 printf("\nAll child processes have been synchronized.\n");
107 printf("Parent process completed.\n");
108
109 return 0;
```

C Tab Width: 8 Ln 110, Col 2 INS

```
Saketh@SakethVutharavalli:~/ossp_practical$ cd ~/ossp_practical
gcc practical4.c -o practical4
./practical4
Parent Process Started
Parent PID: 1664

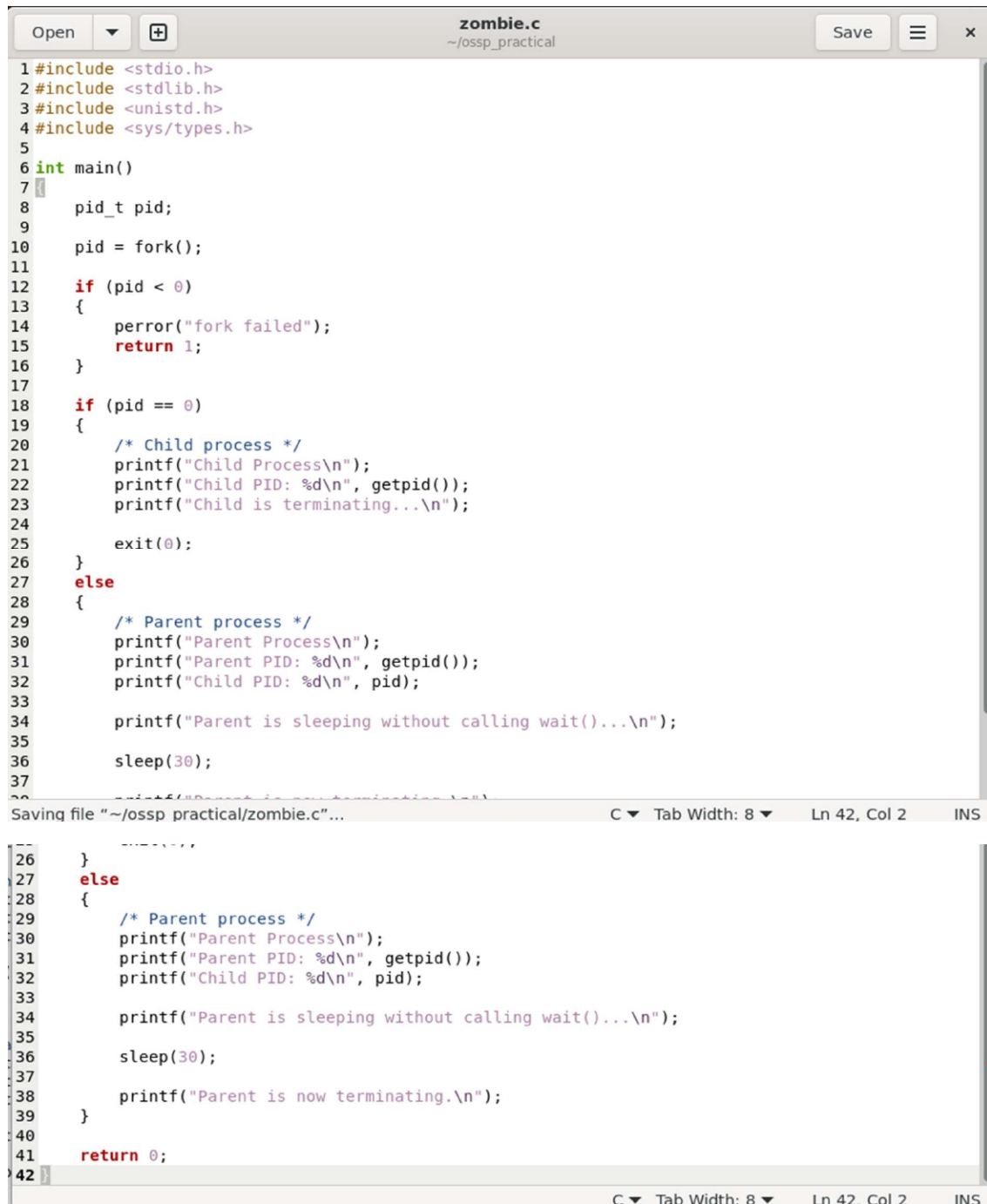
Child 1 started. PID: 1665, PPID: 1664
Child 2 started. PID: 1666, PPID: 1664

Parent waiting using wait()...
Child 3 started. PID: 1667, PPID: 1664
Child 3 completed.
wait(): A child terminated with exit status 30

Parent waiting for Child 2 using waitpid()...
Child 1 completed.
Child 2 completed.
waitpid(): Child 2 terminated with exit status 20
Child 1 terminated with exit status 10

All child processes have been synchronized.
Parent process completed.
Saketh@SakethVutharavalli:~/ossp_practical$
```

Practical – 4 – OS



```
1#include <stdio.h>
2#include <stdlib.h>
3#include <unistd.h>
4#include <sys/types.h>
5
6int main()
7{
8    pid_t pid;
9
10   pid = fork();
11
12   if (pid < 0)
13   {
14       perror("fork failed");
15       return 1;
16   }
17
18   if (pid == 0)
19   {
20       /* Child process */
21       printf("Child Process\n");
22       printf("Child PID: %d\n", getpid());
23       printf("Child is terminating...\n");
24
25       exit(0);
26   }
27   else
28   {
29       /* Parent process */
30       printf("Parent Process\n");
31       printf("Parent PID: %d\n", getpid());
32       printf("Child PID: %d\n", pid);
33
34       printf("Parent is sleeping without calling wait()...\n");
35
36       sleep(30);
37
38       printf("Parent is now terminating.\n");
39   }
40
41   return 0;
42}
```

Practical – 4 – OS

```
Saketh@SakethVutharavalli: ~  
Saketh@SakethVutharavalli:~$ cd ~/ossp_practical  
Saketh@SakethVutharavalli:~/ossp_practical$ ps -o pid,ppid,state,stat  
1720  
    PID    PPID S  STAT CMD  
    1720    1719 Z  Z+   [zombie] <defunct>  
Saketh@SakethVutharavalli:~/ossp_practical$ cat /proc/1720/status | g  
Name:State|Pid|PPid'  
Name:    zombie  
State:   Z (zombie)  
Pid:     1720  
PPid:    1719  
TracerPid:      0  
Saketh@SakethVutharavalli:~/ossp_practical$ █  
  
Parent waiting using wait()...  
Child 3 started. PID: 1667, PPID: 1664  
Child 3 completed.  
wait(): A child terminated with exit status 30  
  
Parent waiting for Child 2 using waitpid()...  
Child 1 completed.  
Child 2 completed.  
waitpid(): Child 2 terminated with exit status 20  
Child 1 terminated with exit status 10  
  
All child processes have been synchronized.  
Parent process completed.  
Saketh@SakethVutharavalli:~/ossp_practical$ gedit zombie.c  
Saketh@SakethVutharavalli:~/ossp_practical$ cd ~/ossp_practical  
gcc zombie.c -o zombie  
Saketh@SakethVutharavalli:~/ossp_practical$ cd ~/ossp_practical  
./zombie  
Parent Process  
Parent PID: 1719  
Child PID: 1720  
Parent is sleeping without calling wait()...  
Child Process  
Child PID: 1720  
Child is terminating...
```

```
*zombie_fixed.c  
~/ossp_practical  
Save  
x  
1 #include <stdio.h>  
2 #include <stdlib.h>  
3 #include <unistd.h>  
4 #include <sys/types.h>  
5 #include <sys/wait.h>  
6  
7 int main()  
8  
9     pid_t pid;  
10    int status;  
11  
12    pid = fork();  
13  
14    if (pid < 0)  
15    {  
16        perror("fork failed");  
17        return 1;  
18    }  
19  
20    if (pid == 0)  
21    {  
22        /* Child process */  
23        printf("Child Process\n");  
24        printf("Child PID: %d\n", getpid());  
25        printf("Child is terminating...\n");  
26        exit(0);  
27    }  
28    else  
29    {  
30        /* Parent process */  
31        printf("Parent Process\n");  
32        printf("Parent PID: %d\n", getpid());  
33        printf("Child PID: %d\n", pid);  
34  
35        printf("Parent is waiting for the child using wait()...\n");  
36  
37        wait(&status);  
38    }  
39  
40    return 0;  
41  
42    }
```

Bracket match found on line: 8 C Tab Width: 8 Ln 50, Col 2 INS

Practical – 4 – OS

```
35
36     printf("Parent is waiting for the child using wait()...\n");
37
38     wait(&status);
39
40     if (WIFEXITED(status))
41     {
42         printf("Child was properly reaped.\n");
43         printf("Child exit status: %d\n", WEXITSTATUS(status));
44     }
45
46     printf("No zombie process remains.\n");
47 }
48
49 return 0;
50
```

C ▾ Tab Width: 8 ▾ Ln 50, Col 2 INS

```
Saketh@SakethVutharavalli:~/ossp_practical$ cd ~/ossp_practical
gcc zombie_fixed.c -o zombie_fixed
Saketh@SakethVutharavalli:~/ossp_practical$ ./zombie_fixed
Parent Process
Parent PID: 1758
Child PID: 1759
Parent is waiting for the child using wait()...
Child Process
Child PID: 1759
Child is terminating...
Child was properly reaped.
Child exit status: 0
No zombie process remains.
Saketh@SakethVutharavalli:~/ossp_practical$
```

```
Saketh@SakethVutharavalli:~/ossp_practical$ ps -o pid,ppid,state,stat,cmd -p
1759
PID      PPID S  STAT CMD
Saketh@SakethVutharavalli:~/ossp_practical$
```